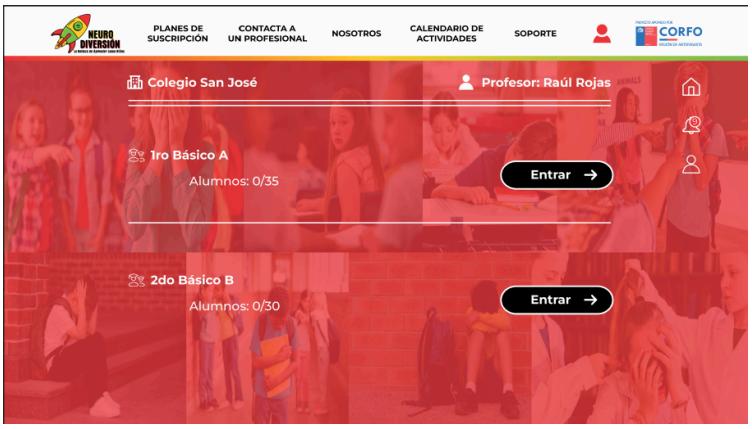
Milestone 3

Observation: The designs in Figma don't have a mobile adaptation. I need you to make a simple adaptation based on your own judgment. Let me know if you can handle this.

1. **Teacher dashboard view:**It is entered when the get_app_context() function returns permissions.view_type = **'teacher_dashboard.** Displays the list of courses related to a user of the teacher type.



   a. To load the information to be displayed, the RPC function **'get_teacher_courses_overview'** must be called:

      i. **Input:** None. The database reads the auth.uid() automatically, so you only need to call the empty function.

      ii. **Output:** A single JSON object with this exact format:

```
{
"teacher_name": "Teacher's Name",
"institution_name": "Institution Name or 'No Institution Assigned'",
  "global_subscription_active": true,
  "total_courses": 2,
  "courses": [
    {
      "course_id": 14,
"course_name": "1st Basic A",
      "student_limit": 30,
      "total_students": 25
    },
    {
      "course_id": 15,
      "course_name": "2do Básico B",
      "student_limit": 20,
      "total_students": 10
    }
  ]
}
```

      iii. **Instructions for positioning the data:**

         1. 'teacher_name':

         2. 'institution_name':

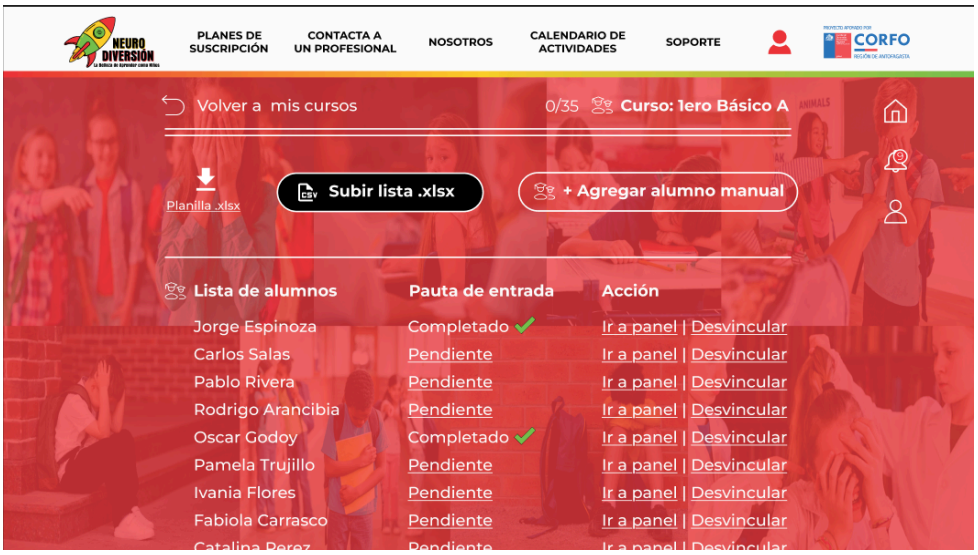
3. ‘course_name’:

4. Alumnos: ‘total_students’/‘student_limit’:

iv. Enter button:

It should redirect to the ‘**course_dashboard**’ page: This interface displays the list of students related to the course, the possibility of uploading students manually and through a .CSV spreadsheet.

2. Teacher course_dashboard



a. To load the information, you must call the RPC function **‘get_course_dashboard(p_course_id)’**

   i. **Input:** ‘course.id’

   ii. **Output:** A single JSON object with this exact format

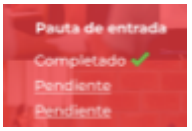

```
{
  "course_id": 14,
"course_name": "1st Basic A",
  "student_limit": 30,
  "total_students": 2,
  "is_plan_active": true,
  "students": [
    {
      "id": "e4f3c... (uuid)",
"full_name": "Ana García",
     "assessment_info": { "instance_id": 105, "status": "PENDING", "current_section": 2 }
    {
      "id": "b1a2d... (uuid)",
      "full_name": "Pedro Soto",
       "assessment_info": { "instance_id": 105, "status": "PENDING", "current_section": 2 }
    }
  ]
}
```

iii. **Instructions for positioning the data:**

1. ‘course_name’:

2. ‘total_students’/’student_limit’:

3. ‘full_name’:

4. ‘is_assessment_completed’: 

   a. This column can display 2 options depending on whether the result of ‘is_assessment_completed’ is ‘COMPLETED’ or not:

      i. If it is ‘COMPLETED’, plain text + a ticket icon will be displayed:

      ii. If it is not 'COMPLETED' ('IN_PROGRESS' OR 'PENDING') a hyperlink is displayed that redirects to the path 'assessment/'instance_id'/'current_section'

iv. **Features:**

1. **Redirect to dashboard of ‘Student’:**

   a. **Redirect to the URL:**'nna/[nna_id]

2. **'Unlink' button:** Remove a student from the list.

   a. **If "is_plan_active": false, the action should be blocked.**

b. **Pressing it should display the modal:**

c. **Action confirmation:** By pressing the red button Call the function**: 'remove_student_from_course'**

   i. **Input:** studentId, courseId

3. **Download template**

   a. Position the file in the '/public' folder using a native <a> tag with the download attribute, so that the browser downloads it instead of trying to open it.

4. **Reading '.xlsx' file:**For bulk student uploads, the database will handle all complex validations. Your responsibility is simply to read the .xlsx file, remove empty rows, format the date to ISO, and send me a JSON file.

   a. **If "is_plan_active": false, the action should be blocked.**

   b. The first 4 rows are headers/instructions. The actual data starts in row 5 (array index 4).

   c. Columns: [0] first_name, [1] last_name, [2] sex, [3] birthdate, [4] email

5. **File upload confirmation**: Call the function '**bulk_import_students**(course_id, students (jsonb))

This button only appears when a document has been uploaded.

a. Function output data:

i. Success story:

```
{
  "success": true,
"message": "Import completed successfully.",
  "details": {
    "students_added": 5,
    "parents_directly_linked": 1,
    "invitations_created": 2
  }
}
```

ii. Case failure:

```
{
  "success": false,
"error": "You do not have permission to modify this course."
}
```

iii. Make sure to evaluate if (!data.success) in addition to checking for Supabase's native error, and display the data.error in the interface.

6. Add student manually:

a. **If "is_plan_active": false, the action should be blocked.**

b. You must display the following modal:

i. It is similar to the existing form, but an additional email field is added (Validate format).

c. Upon confirmation, the following function must be called: **‘add_student_to_course’**

i. Input data:

```
supabase.rpc("add_student_to_course", {
   p_course_id: 1,
   p_first_name: studentFormData.firstName.trim(),
   p_last_name: studentFormData.lastName.trim(),
   p_birthdate: studentFormData.birthdate, // Formato 'YYYY-MM-DD'
p_sex: studentFormData.sex, // 'm' or 'f' (depending on the database ENUM)
   p_parent_email: studentFormData.parentEmail?.trim()?.toLowerCase() || null
 });
```

ii. Output data (JSON):

1. Success story

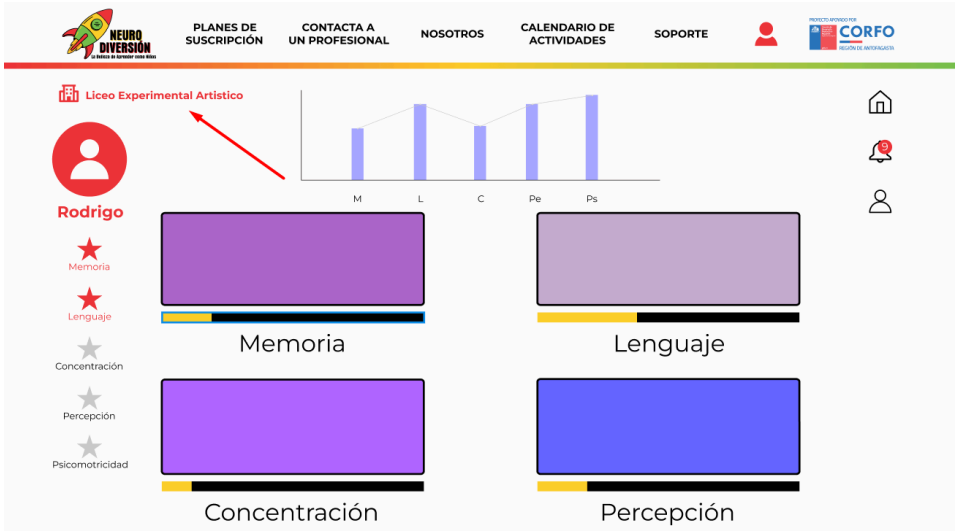
```
{
   "success": true,
"message": "Student created successfully.",
   "student_id": "uuid-del-niño",
   "assessment_instance_id": 123
 }
```

2. Failed case

```
{
   "success": false,
"error": "Error creating the student."
 }
```

3. **Other modifications**

a. **Add visual component to view ‘nna/[id]’**



i. The text is obtained from **‘get_nna_dashboard_overview’** → **‘student’ object** → **‘institution_name’.** If it is null, do not display anything.